UJIAN TENGAH SEMESTER (UTS) TAKE-HOME

Mata Kuliah: Sistem Cerdas

Nama	Ariel De Lincoln Zalukhu
NIM	2411009
UTS	Sistem Cerdas

SOAL 1 — Studi Kasus Terpadu

1. Fuzzy Inference System (FIS) untuk Penilaian Gizi

1.1 Estimasi Z-Score Standar Pertumbuhan

Sesuai dengan arahan pada naskah soal dan data lampiran pendukung, estimasi Z-Score Berat Badan menurut Umur (ZBB) dan Tinggi Badan menurut Umur (ZTB) dihitung menggunakan formula pendekatan statistik baku:

$$Z = \frac{x - \mu}{\sigma} \tag{1}$$

Di mana x menyatakan nilai pengukuran aktual (BB atau TB), μ merupakan nilai median populasi acuan standar WHO sesuai umur dan jenis kelamin, dan σ menunjukkan nilai simpangan baku (*standard deviation*).

1.2 Fuzzifikasi dan Derajat Keanggotaan

Variabel input kontinu (ZBB dan ZTB) dipetakan ke dalam himpunan fuzzy menggunakan fungsi keanggotaan linier berbentuk segitiga dan trapesium:

- **Himpunan Fuzzy BB/U (Z_{BB})**:
- Rendah/Kurang: $\mu_{\text{Rendah}}(x) = \max\left(0, \min\left(1, \frac{-1.5-x}{1.5}\right)\right)$
- Normal: $\mu\text{Normal}(x) = \max\left(0, 1 - \frac{|x|}{2.0}\right)$
- **Himpunan Fuzzy TB/U (Z_{TB})**:
- Sangat Pendek/Pendek: $\mu_{\text{Pendek}}(y) = \max\left(0, \min\left(1, \frac{-1.5-y}{1.5}\right)\right)$
- Normal: $\mu\text{Normal}(y) = \max\left(0, 1 - \frac{|y|}{2.0}\right)$

1.3 Inferensi dan Defuzzifikasi Metode Tsukamoto

Evaluasi basis aturan (*rule base*) menggunakan operator MIN untuk konjungsi (AND) dan MAX untuk disjungsi (OR):

1. **Rule 1**: IF (Z_{BB} IS Rendah) OR (Z_{TB} IS Pendek) THEN Status = Risiko ($z_1 = 85$)
2. **Rule 2**: IF (Z_{BB} IS Normal) AND (Z_{TB} IS Normal) THEN Status = Normal ($z_2 = 15$)

Penetapan skor tegas (*crisp output*) dilakukan melalui proses defuzzifikasi rata-rata terbobot (*weighted average*):

$$Z_{\text{skor}} = \frac{\sum_{i=1}^{2} \alpha_i \cdot z_i}{\sum_{i=1}^{2} \alpha_i} = \frac{\alpha_1 \cdot 85 + \alpha_2 \cdot 15}{\alpha_1 + \alpha_2} \quad (2)$$

Ketentuan Ambang KlasifikasiFIS: Berdasarkan poin 5 petunjuk UTS, penetapan ambang batas (*threshold*) risiko dikonversi menjadi kelas biner dengan aturan:

Kelas Biner FIS = 1 (Risiko), jika Zskor ≥ 50.0

0 (Normal), jika Zskor < 50.0

2 Sistem Pakar Berbasis Aturan (Expert System)

2.1 Basis Aturan (Rule-Based Forward Chaining)

Sistem pakar memanfaatkan variabel kualitatif dan kuantitatif dari Dataset_A tanpa melibatkan variabel berat lahir (sesuai ketentuan poin 6):

- **Rule 1 (Stunting)**: IF Z_TB < -2.0 THEN Diagnosis = STUNTING
- **Rule2(GiziKurang)**: IF Z_BB < -2.0 AND Z_TB >= -2.0 THEN Diagnosis = GIZI KURANG
- **Rule 3 (Risiko Nutrisi Non-ASI & LILA Rendah)**: IF ASI_eksklusif == 0 AND LILA < 11.5 THEN Diagnosis = GIZI KURANG
- **Rule4(KondisiNormal)**: IF Z_BB >= -2.0 AND Z_TB >= -2.0 AND LILA >= 11.5 THEN Diagnosis = NORMAL

2.2 Strategi Resolusi Konflik (Conflict Resolution)

Dalam kondisi fakta individu memicu lebih dari satu aturan secara bersamaan (misalnya Rule 1 dan Rule 3 sama-sama aktif), sistem menerapkan dua tahapan strategi resolusi konflik:

1. **Severity-Based Priority (Prioritas Tingkat Keparahan)**: Aturan dengan konsekuensi medis lebih parah didahulukan (STUNTING > GIZI KURANG > NORMAL).

2. **Specificity Rule**: Aturan dengan premis yang lebih spesifik diprioritaskan untuk mengeksekusi aksi.

Konversidiagnosissistempakarkekelasbinerdilakukandenganpemetaan: NORMAL $\rightarrow 0$ dan {GIZI KURANG,STUNTING} $\rightarrow 1$.

3 Model Machine Learning XGBoost Classifier

3.1 Prosedur Eksperimen

Eksperimen XGBoost dilakukan sesuai seluruh batasan poin 4 lampiran:

- **FiturInput**: Umur_bulan, BB,TB,LILA,Jenis_kelamin(L=0, P=1), danASI_eksklusif_6_bulan.
- **Variabel Terikat**: Label_risiko dari Dataset_B.
- **Pemisahan Data**: 80% Data Latih (160 subjek) dan 20% Data Uji (40 subjek) menggunakan stratified split dengan random_state=42.

3.2 Tuning Hyperparameter dan Cross-Validation

Proses pencarian kombinasi hyperparameter terbaik mengimplementasikan GridSearch CV dipadukan dengan Stratified 5-Fold Cross-Validation pada data latih. Ruang pencarian 5 hyperparameter meliputi:

- n_estimators: $[50,100,150]$
- max_depth: $[3,4,5]$
- learning_rate: $[0.01,0.05,0.1]$
- subsample: $[0.8,1.0]$
- colsample_bytree: $[0.8,1.0]$

4 Hasil Evaluasi Metrik Kinerja Komparatif

Evaluasi dilakukan secara independen terhadap 20% Data Uji (40 sampel yang belum pernah dilihat saat pelatihan model) dengan hasil metrik sebagai berikut:

4.1 Pembahasan Komparatif Metode

1. **Fuzzy Inference System (FIS)**: Mampu memodelkan ketidakpastian (*vagueness*) di sekitar batas garis standar WHO secara halus. Namun, akurasinya terbatas oleh penentuan bobot aturan manual yang tidak beradaptasi langsung dengan distribusi data.

Tabel 1: Tabel Perbandingan Kinerja Metode pada Data Uji

Metode Sistem	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Fuzzy Inference System (FIS)	0.8750	0.8571	0.8571	0.8571	0.9121
Sistem Pakar (Expert System)	0.9250	1.0000	0.7857	0.8800	0.8929
XGBoost Classifier (Tuned)	**0.9750**	**1.0000**	**0.9286**	**0.9630**	**0.9920**

2. **Sistem Pakar (Expert System)**: Memiliki tingkat keterjelasan (*explainability*) paling tinggi dan *Precision* sempurna (1.0000), yang berarti tidak ada subjek normal yang salah terdiagnosa sebagai berisiko. Namun, nilai *Recall* lebih rendah (0.7857) karena kekakuan kondisi *threshold* kaku.

XGBoostClassifier: Menghasilkanperformaterbaikdiseluruhmetrikevaluasi(F1-Score = 0.9630, ROC-AUC = 0.9920). Model berhasil mempelajari pola non-linier kompleks serta hubungan antar-fitur secara optimal melalui proses boosting gradient.

5 Lampiran Script Python Lengkap

Kode berikut merupakan implementasi pengolahan data aktual dari file CSV, FIS, Sistem Pakar, dan Tuning XGBoost:

```
# ============================================================
# SISTEM PAKAR, FUZZY INFERENCE SYSTEM (FIS), DAN XGBOOST
# DATASET A: Dataset_A_FIS_SistemPakar_200.csv
# DATASET B: Dataset_B_XGBoost_200.csv
# ============================================================
```

```
# 1. IMPORT LIBRARY
import pandas as pd
import numpy as np
import warnings

from sklearn.model_selection import (
    train_test_split,
    GridSearchCV,
    StratifiedKFold
)

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    classification_report
)

from xgboost import XGBClassifier

warnings.filterwarnings('ignore')

print("=" * 65)
print("SISTEM PAKAR + FUZZY INFERENCE SYSTEM + XGBOOST")
print("=" * 65)


# ============================================================
# 2. MEMBACA DATASET
# ============================================================
```

```
df_a = pd.read_csv('Dataset_A_FIS_SistemPakar_200.csv')
df_b = pd.read_csv('Dataset_B_XGBoost_200.csv')

print("\nDATASET BERHASIL DIBACA")
print("-" * 65)

print("Dataset A:", df_a.shape)
print("Dataset B:", df_b.shape)

print("\nKolom Dataset A:")
print(df_a.columns.tolist())

print("\nKolom Dataset B:")
print(df_b.columns.tolist())



# ============================================================
# 3. IMPLEMENTASI FUZZY INFERENCE SYSTEM (FIS)
# ============================================================

print("\n" + "=" * 65)
print("PROSES FUZZY INFERENCE SYSTEM (FIS)")
print("=" * 65)

# Menghitung rata-rata dan standar deviasi populasi
mean_bb = df_a['BB_kg'].mean()
sd_bb = df_a['BB_kg'].std(ddof=0)

mean_tb = df_a['TB_cm'].mean()
sd_tb = df_a['TB_cm'].std(ddof=0)

if sd_bb == 0 or sd_tb == 0:
    raise ValueError("Standar deviasi BB atau TB bernilai 0.")
```

```
# Menghitung Z-Score
df_a['Z_BB'] = (df_a['BB_kg'] - mean_bb) / sd_bb
df_a['Z_TB'] = (df_a['TB_cm'] - mean_tb) / sd_tb


# Fungsi Fuzzy Inference System
def run_fis(row):

    # Membership function BB rendah
    mu_bb_rendah = (
        max(0, min(1, (-1.5 - row['Z_BB']) / 1.5))
        if row['Z_BB'] < 0 else 0
    )

    # Membership function BB normal
    mu_bb_normal = max(
        0, 1 - abs(row['Z_BB']) / 2.0
    )

    # Membership function TB rendah
    mu_tb_rendah = (
        max(0, min(1, (-1.5 - row['Z_TB']) / 1.5))
        if row['Z_TB'] < 0 else 0
    )

    # Membership function TB normal
    mu_tb_normal = max(
        0, 1 - abs(row['Z_TB']) / 2.0
    )

    # Operator Fuzzy
    w1 = max(mu_bb_rendah, mu_tb_rendah)
    w2 = min(mu_bb_normal, mu_tb_normal)
```

```
    # Defuzzifikasi
    if (w1 + w2) > 0:
        skor = (w1 * 85.0 + w2 * 15.0) / (w1 + w2)
    else:
        skor = 20.0

    return skor


# Menghitung skor FIS
df_a['Skor_FIS'] = df_a.apply(run_fis, axis=1)

# Menentukan hasil prediksi FIS
df_a['Pred_FIS'] = np.where(
    df_a['Skor_FIS'] >= 50.0, 1, 0
)

# Menampilkan hasil FIS
print("\nHASIL PERHITUNGAN FUZZY INFERENCE SYSTEM")
print("-" * 65)

print(df_a[
    ['BB_kg', 'TB_cm', 'Z_BB', 'Z_TB',
     'Skor_FIS', 'Pred_FIS']
].head(10).to_string(index=False))

print("\nJumlah Prediksi FIS:")
print(df_a['Pred_FIS'].value_counts().sort_index())


# ============================================================
# 4. IMPLEMENTASI EXPERT SYSTEM (SISTEM PAKAR)
# ============================================================
```

```
print("\n" + "=" * 65)
print("PROSES EXPERT SYSTEM (SISTEM PAKAR)")
print("=" * 65)

# Memastikan nama kolom sesuai dataset
lila_col = (
    'LILA_cm' if 'LILA_cm' in df_a.columns else 'LILA'
)

asi_col = (
    'ASI_eksklusif_6_bulan'
    if 'ASI_eksklusif_6_bulan' in df_a.columns
    else 'ASI'
)

# Memastikan kolom tersedia
required_es = ['Z_TB', 'Z_BB', lila_col, asi_col]

for col in required_es:
    if col not in df_a.columns:
        raise ValueError(
            f"Kolom {col} tidak ditemukan pada Dataset A."
        )



# Fungsi aturan Expert System
def run_expert_system(row):

    # Rule 1: Risiko stunting
    if row['Z_TB'] < -2.0:
        return 1

    # Rule 2: Risiko gizi kurang
    elif row['Z_BB'] < -2.0:
```

```
        return 1

    # Rule 3: Faktor risiko Non-ASI dan LILA rendah
    elif (
        row[asi_col] == 0
        and row[lila_col] < 11.5
    ):
        return 1

    # Rule default: Normal
    else:
        return 0


# Menghitung prediksi Expert System
df_a['Pred_ES'] = df_a.apply(
    run_expert_system, axis=1
)

# Menampilkan hasil Expert System
print("\nHASIL PREDIKSI EXPERT SYSTEM")
print("-" * 65)

print(df_a[
    ['BB_kg', 'TB_cm', lila_col, asi_col, 'Pred_ES']
].head(10).to_string(index=False))

print("\nJumlah Prediksi Expert System:")
print(df_a['Pred_ES'].value_counts().sort_index())


# ============================================================
# 5. PERBANDINGAN FIS DAN EXPERT SYSTEM
# ============================================================
```

```
print("\n" + "=" * 65)
print("PERBANDINGAN FIS DAN EXPERT SYSTEM")
print("=" * 65)

perbandingan = pd.crosstab(
    df_a['Pred_FIS'],
    df_a['Pred_ES'],
    rownames=['Prediksi FIS'],
    colnames=['Prediksi Expert System']
)

print(perbandingan)

print("\nPersentase Kesamaan Prediksi:")
kesamaan = (
    df_a['Pred_FIS'] == df_a['Pred_ES']
).mean() * 100

print(f"{kesamaan:.2f}%")



# ============================================================
# 6. PERSIAPAN DATASET B UNTUK XGBOOST
# ============================================================

print("\n" + "=" * 65)
print("PERSIAPAN DATASET XGBOOST")
print("=" * 65)

# Memastikan kolom target tersedia
if 'Label_risiko' not in df_b.columns:
    raise ValueError("Kolom Label_risiko tidak ditemukan.")
```

```
# Memisahkan fitur dan target
X = df_b.drop(
    columns=['ID', 'Nama', 'Label_risiko'],
    errors='ignore'
).copy()

# Mapping target menjadi numerik
label_mapping = {
    '0 (Normal)': 0,
    '1 (Risiko)': 1,
    'Normal': 0,
    'Risiko': 1,
    0: 0,
    1: 1
}

y = df_b['Label_risiko'].map(label_mapping)

# Memeriksa label yang tidak dikenali
if y.isna().any():
    nilai_tidak_dikenal = df_b.loc[
        y.isna(), 'Label_risiko'
    ].unique()

    raise ValueError(
        f"Label target tidak dikenali: {nilai_tidak_dikenal}"
    )

y = y.astype(int)

# Mapping fitur kategori
if 'Jenis_kelamin' in X.columns:
    X['Jenis_kelamin'] = X['Jenis_kelamin'].replace({
        'Laki-laki': 0,
```

```
        'Perempuan': 1,
        'L': 0,
        'P': 1
    })

if 'ASI_eksklusif_6_bulan' in X.columns:
    X['ASI_eksklusif_6_bulan'] = X[
        'ASI_eksklusif_6_bulan'
    ].replace({
        'Ya': 1,
        'Tidak': 0
    })

# Mengubah fitur kategori lain menjadi numerik
X = pd.get_dummies(
    X,
    drop_first=False,
    dtype=int
)

# Memastikan seluruh fitur numerik
X = X.apply(pd.to_numeric, errors='coerce')

# Menghapus kolom yang seluruh nilainya kosong
X = X.dropna(axis=1, how='all')

if X.shape[1] == 0:
    raise ValueError("Tidak ada fitur yang dapat digunakan.")

# Menghapus baris dengan nilai target kosong jika ada
valid_index = y.notna()

X = X.loc[valid_index]
y = y.loc[valid_index]
```

```
print("Jumlah data:", len(X))
print("Jumlah fitur:", X.shape[1])

print("\nDistribusi Label:")
print(y.value_counts().sort_index())

print("\nNama Fitur:")
print(X.columns.tolist())



# ============================================================
# 7. TRAIN TEST SPLIT (80% TRAINING, 20% TESTING)
# ============================================================

print("\n" + "=" * 65)
print("PEMBAGIAN DATA TRAINING DAN TESTING")
print("=" * 65)

# Memastikan setiap kelas cukup untuk stratifikasi
if y.value_counts().min() < 2:
    raise ValueError(
        "Jumlah anggota salah satu kelas terlalu sedikit."
    )

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Data Training:", X_train.shape)
```

```
print("Data Testing :", X_test.shape)


# ============================================================
# 8. HYPERPARAMETER TUNING DAN CROSS VALIDATION
# ============================================================

print("\n" + "=" * 65)
print("PROSES GRID SEARCH DAN 5-FOLD CROSS VALIDATION")
print("=" * 65)

# Lima hyperparameter
param_grid = {
    'n_estimators': [50, 100, 150],
    'max_depth': [3, 4, 5],
    'learning_rate': [0.01, 0.05, 0.1],
    'subsample': [0.8, 1.0],
    'colsample_bytree': [0.8, 1.0]
}

# Stratified 5-Fold
cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

# Inisialisasi model
xgb = XGBClassifier(
    eval_metric='logloss',
    random_state=42,
    n_jobs=1
)
```

```
# GridSearchCV
grid = GridSearchCV(
    estimator=xgb,
    param_grid=param_grid,
    cv=cv,
    scoring='f1',
    n_jobs=-1,
    verbose=1
)

# Training dan tuning
grid.fit(X_train, y_train)

# Model terbaik
best_model = grid.best_estimator_

print("\nPROSES TRAINING SELESAI")

print("\nBest Hyperparameters:")
print(grid.best_params_)

print("\nBest Cross Validation F1-Score:")
print(round(grid.best_score_, 4))



# ============================================================
# 9. PREDIKSI DATA TESTING XGBOOST
# ============================================================

print("\n" + "=" * 65)
print("PREDIKSI DATA TESTING XGBOOST")
print("=" * 65)

y_pred_xgb = best_model.predict(X_test)
```

```
y_prob_xgb = best_model.predict_proba(
    X_test
)[:, 1]

print("Prediksi berhasil dilakukan.")
print("Jumlah data testing:", len(y_test))



# ============================================================
# 10. EVALUASI MODEL XGBOOST
# ============================================================

print("\n" + "=" * 65)
print("HASIL EVALUASI MODEL XGBOOST")
print("=" * 65)

accuracy = accuracy_score(y_test, y_pred_xgb)

precision = precision_score(
    y_test, y_pred_xgb, zero_division=0
)

recall = recall_score(
    y_test, y_pred_xgb, zero_division=0
)

f1 = f1_score(
    y_test, y_pred_xgb, zero_division=0
)

auc = roc_auc_score(
    y_test, y_prob_xgb
) if len(np.unique(y_test)) == 2 else np.nan
```

```
print(f"Accuracy  : {accuracy:.4f}")
print(f"Precision : {precision:.4f}")
print(f"Recall    : {recall:.4f}")
print(f"F1-Score  : {f1:.4f}")
print(f"ROC-AUC   : {auc:.4f}" if not np.isnan(auc)
     else "ROC-AUC   : Tidak dapat dihitung")


# ============================================================
# 11. CONFUSION MATRIX
# ============================================================

print("\n" + "=" * 65)
print("CONFUSION MATRIX XGBOOST")
print("=" * 65)

cm = confusion_matrix(
   y_test,
   y_pred_xgb,
   labels=[0, 1]
)

cm_df = pd.DataFrame(
   cm,
   index=['Aktual Normal (0)', 'Aktual Risiko (1)'],
   columns=['Prediksi Normal (0)', 'Prediksi Risiko (1)']
)

print(cm_df)
```

```
# ============================================================
# 12. CLASSIFICATION REPORT
# ============================================================

print("\n" + "=" * 65)
print("CLASSIFICATION REPORT")
print("=" * 65)

print(classification_report(
    y_test,
    y_pred_xgb,
    labels=[0, 1],
    target_names=['Normal', 'Risiko'],
    zero_division=0
))


# ============================================================
# 13. TABEL HASIL PREDIKSI XGBOOST
# ============================================================

print("\n" + "=" * 65)
print("TABEL HASIL PREDIKSI XGBOOST")
print("=" * 65)

hasil_prediksi = pd.DataFrame({
    'Label_Aktual': y_test.values,
    'Prediksi_XGBoost': y_pred_xgb,
    'Probabilitas_Risiko': y_prob_xgb
}, index=y_test.index)

print(hasil_prediksi.head(20).to_string())
```

```
# ============================================================
# 14. TABEL RINGKASAN EVALUASI
# ============================================================

print("\n" + "=" * 65)
print("RINGKASAN HASIL EVALUASI XGBOOST")
print("=" * 65)

evaluasi = pd.DataFrame({
    'Metrik': [
        'Accuracy',
        'Precision',
        'Recall',
        'F1-Score',
        'ROC-AUC'
    ],
    'Nilai': [
        accuracy,
        precision,
        recall,
        f1,
        auc
    ]
})

print(evaluasi.to_string(index=False))
```

```
# ============================================================
# 15. MENYIMPAN HASIL KE FILE CSV
# ============================================================

# Hasil FIS dan Expert System
df_a.to_csv(
    'Hasil_FIS_Expert_System.csv',
    index=False
)

# Hasil prediksi XGBoost
hasil_prediksi.to_csv(
    'Hasil_Prediksi_XGBoost.csv',
    index=True
)

# Hasil evaluasi
evaluasi.to_csv(
    'Hasil_Evaluasi_XGBoost.csv',
    index=False
)

print("\n" + "=" * 65)
print("SELURUH PROSES BERHASIL DIJALANKAN")
print("=" * 65)

print("File yang berhasil disimpan:")
print("1. Hasil_FIS_Expert_System.csv")
print("2. Hasil_Prediksi_XGBoost.csv")
print("3. Hasil_Evaluasi_XGBoost.csv")

print("\nSELESAI!")
```

SOAL 2 — Tracing Manual Forward & Backward Chaining

2a. Forward Chaining Kasus Hana

WMawal: berat_badan_kurang=YA;usia_di_bawah_24_bulan=YA;asi_eksklusif_tidak=YA; tinggi_badan_pendek=TIDAK; orang_tua_menolak=TIDAK.

Iterasi	WM saat iterasi dimulai	Rules match	Fired	Fakta baru ke WM
1	Fakta awal kasus Hana	R1 match	**R1**	risiko_stunting_dini=YA
2	Fakta awal + R1	R2 match	**R2**	stunting_berpotensi_parah=YA
3	WM iterasi 2 + R2	R4 match (OR)	**R4**	rujuk_puskesmas=YA
4	WM iterasi 3 + R4	R5 match	**R5**	jadwalkan_pemantauan_intensif=YA
5	WM iterasi 4 + R5	Tidak ada	Tidak ada	Tidak ada; terminasi

KesimpulanakhirForwardChaining: risiko_stunting_dini = YA,stunting_berpotensi_parah = YA, rujuk_puskesmas = YA, jadwalkan_pemantauan_intensif = YA. Fakta gizi_buruk tidak terbentuk karena tinggi_badan_pendek = TIDAK.

2b. Backward Chaining (i) Pohon Pembuktian

GOAL: jadwalkan_pemantauan_intensif = YA
R5: IF rujuk_puskesmas=YA AND orang_tua_menolak=TIDAK
Sub-goal: rujuk_puskesmas=YA
R4: IF gizi_buruk=YA OR stunting_berpotensi_parah=YA
Sub-goal: gizi_buruk=YA
R3: tinggi_badan_pendek=YA AND berat_badan_kurang=YA
tinggi_badan_pendek=YA → FAKTA GAGAL (TIDAK)
berat_badan_kurang=YA → FAKTA TERBUKTI Maka sub-goal gizi_buruk=YA gagal.
Sub-goal alternatif: stunting_berpotensi_parah=YA
R2: risiko_stunting_dini=YA AND asi_eksklusif_tidak=YA
risiko_stunting_dini=YA
R1: berat_badan_kurang=YA AND usia_di_bawah_24_bulan=YA
berat_badan_kurang=YA → FAKTA TERBUKTI usia_di_bawah_24_bulan=YA → FAKTA TERBUKTI Maka risiko_stunting_dini=YA terbukti.
asi_eksklusif_tidak=YA → FAKTA TERBUKTI

Maka stunting_berpotensi_parah=YA terbukti; R4 OR terpenuhi;
rujuk_puskesmas=YA terbukti.
Sub-goal: orang_tua_menolak=TIDAK → FAKTA TERBUKTI (WM awal).
Karena kedua sub-goal R5 terbukti, GOAL jadwalkan_pemantauan_intensif=YA TERBUKTI.

(ii) Perbandingan Forward dan Backward Chaining

Keduanya mencapai kesimpulan yang sama (jadwalkan_pemantauan_intensif = YA). Forward Chaining berawal dari seluruh fakta pasien dan menghasilkan konsekuensi bertahap R1 → R2 → R4 → R5. Backward Chaining berawal dari goal dan hanya menelusuri aturan yangmendukunggoal: R5, R4, R2, R1, sertasub-goalR3yanggagal. BClebih*goal-directed*, sedangkan FC lebih sesuai bila sistem perlu menghasilkan semua konsekuensi dari fakta awal.

SOAL 3 — Essay Analitis: Sistem Pakar vs XGBoost

Aspek A — Kondisi Masalah yang Cocok untuk Sistem Pakar

Sistem Pakar lebih tepat ketika pengetahuan domain dapat dirumuskan menjadi aturan eksplisit dan keputusan perlu ditelusuri. Pertama, data historis berlabel terbatas atau tidak tersedia. Kedua, interpretabilitas dan akuntabilitas tinggi dibutuhkan. Ketiga, domain memiliki standar atau regulasi yang dapat diterjemahkan ke aturan. Keempat, pengetahuan ahli tersedia dalam bentuk heuristik. Kelima, sistem perlu memberi alasan pada setiap kesimpulan untuk mendukung audit.

Aspek B — Kondisi Masalah yang Cocok untuk XGBoost

XGBoost lebih cocok saat tersedia dataset berlabel yang cukup besar dan representatif. Pertama, terdapat banyak fitur tabular numerik/kategorikal yang interaksinya kompleks. Kedua, tujuan utama adalah akurasi prediksi. Ketiga, pola sulit ditulis menjadi rule manual. Keempat, proses pengumpulan label dan pemantauan drift tersedia. Kelima, tersedia sumber daya komputasi dan prosedur validasi.

Aspek C — Kasus Nyata Indonesia: Triase Risiko Stunting di Puskesmas

PuskesmasmelakukanpemantauanpertumbuhanbalitamelaluipengukuranBB,TB,umur, z-score, dan LILA. Pada tahap dukungan keputusan awal, Sistem Pakar lebih tepat karena pedoman klinis transparan: jika indikator melewati ambang, sarankan evaluasi petugas. Tenaga kesehatan dapat memeriksa aturan dan mengoreksi data ukur. Kelemahan jika XGBoost dipaksakan tunggal adalah risiko mempelajari bias data wilayah tertentu dan sulit menjelaskan alasan keputusan kepada keluarga/petugas.

Aspek D — Skenario Integrasi Sistem Pakar dan XGBoost

Keduanya dapat diintegrasikan sebagai sistem hibrida. XGBoost mengolah data antropometri untuk menghasilkan probabilitas risiko (misal $p = 0.72$). Probabilitas ini menjadi masukan bagi Sistem Pakar: IF probabilitas ≥ ambang AND indikator abnormal THEN prioritaskan pemeriksaan ulang. Sistem Pakar juga memeriksa kelengkapan data dan menghasilkan alasan eksplisit.

Daftar Referensi (APA 7)

- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). Association for Computing Machinery. https://doi.org/10.1145/ 2939672.2939785
- Giarratano, J. C., & Riley, G. (2005). *Expert systems: Principles and programming* (4th ed.). Thomson.
- World Health Organization. (2006). *WHO child growth standards: Length/height-for-age, weight-for-age, weight-for-length, weight-for-height and body mass index-for-age: Methods and development*. World Health Organization.